

Autocomplete Repetitive Stroking with Image Guidance

Yilan Chen
City University of Hong Kong
Hong Kong
yil.ellen.chan@gmail.com

Li-Yi Wei
Adobe Research
United States of America
review@liyiwei.org

Kin Chung Kwan
University of Konstanz, City University of Hong Kong
Germany, Hong Kong
kckwan@ieee.org

Hongbo Fu
City University of Hong Kong
Hong Kong
fuplus@gmail.com

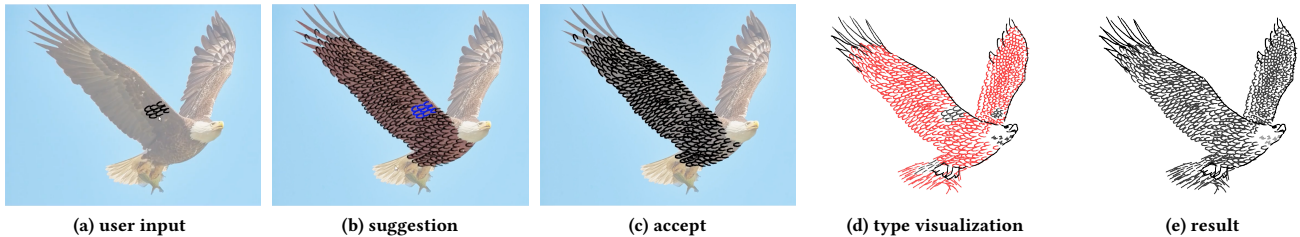


Figure 1: Example of our system workflow. A user draws some feathers over the wing region of a reference image (a) while our system predicts what she might draw next (b) (blue strokes: inferred exemplars; pale red region: inferred target region; semi-transparent strokes: system suggestions), which is then accepted by the user (c) (black strokes: user inputs or accepted suggestions in this scene). (d) visualizes all the manually drawn content in black (88 strokes) and autocompleted content in red (604 strokes). (e) shows the final result. Reference photo by Mathew Schwartz from Unsplash.

ABSTRACT

Image-guided drawing can compensate for the lack of skills but often requires a significant number of repetitive strokes to create textures. Existing automatic stroke synthesis methods are usually limited to predefined styles or require indirect manipulation that may break the spontaneous flow of drawing. We present a method to autocomplete repetitive short strokes during users' normal drawing process. Users can draw over a reference image as usual. At the same time, our system silently analyzes the input strokes and the reference to infer strokes that follow users' input style when certain repetition is detected. Our key idea is to jointly analyze image regions and operation history for detecting and predicting repetitions. The proposed system can reduce tedious repetitive inputs while being fully under user control.

CCS CONCEPTS

• Computing methodologies → Computer graphics; • Human-centered computing → User interface design.

KEYWORDS

Interactive System; Autocompletion; Digital Drawing; Prediction; Texture Synthesis

ACM Reference Format:

Yilan Chen, Kin Chung Kwan, Li-Yi Wei, and Hongbo Fu. 2021. Autocomplete Repetitive Stroking with Image Guidance. In *SIGGRAPH Asia 2021 Technical Communications (SA '21 Technical Communications)*, December 14–17, 2021, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3478512.3488595>

1 INTRODUCTION

Drawing is a common form of artistic expression. By varying the stroke, texture, and shading, artists can create drawings with various styles [Dunn 2015]. Yet, it remains a largely manual process that may require significant artistic expertise and repetitive manual labor.

Various methods have been proposed to synthesize user-initiated repetitive strokes [Kazi et al. 2012; Xing et al. 2014] to reduce the manual labor. However, such methods still require sufficient artistic experience for high-level picture composition. One common way to overcome this skill barrier is to use a reference photo as a scaffold for drawing. With a given reference, many methods exist to automate the synthesis of details, such as contours, textures, or strokes [Jing et al. 2017; Kyprianidis et al. 2013], with the effects tunable via input parameters or exemplars. However, since these algorithms largely predefine the behaviors, their results may look canned and cannot give users a sense of ownership. Furthermore, tweaking parameters

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA '21 Technical Communications, December 14–17, 2021, Tokyo, Japan

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9073-6/21/12...\$15.00

<https://doi.org/10.1145/3478512.3488595>

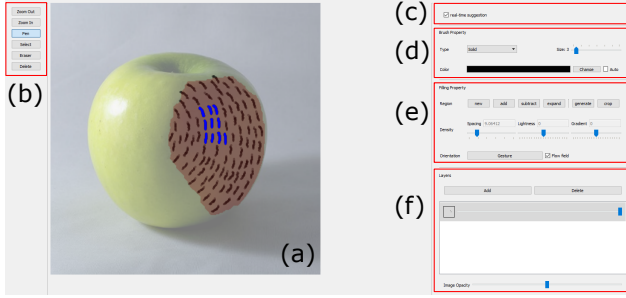


Figure 2: User interface, consisting of a central drawing canvas (a), a toolbar for drawing and selection (b), a toggle-switch of the autocomplete mode (c), a brush property toolbar (d), a filling property toolbar (e), and a layers panel (f). Reference image courtesy of Jennifer Thomas Wick, used with permission.

or providing exemplars can break the spontaneous flow of direct drawing manipulation.

Manual drawing provides sufficient freedom for individual expressing even when scaffolded with a reference image, and its typical interface (e.g., brush, eraser) is familiar to general users. Thus, we aim to enhance the manual drawing process and the typical UI design by automating tedious repetitions. Our idea is to bridge the two extremes: *manual drawing*, which allows full control but can be tedious; and *image-based algorithmic synthesis*, which saves efforts but provides limited user control and interactivity.

Like typical digital drawing applications, users can draw freely on a reference image with our system (Figure 2). Meanwhile, our system analyzes the relationships between user inputs and the reference image, detects potential repetitions, and suggests what users might want to draw next. Users can accept, reject, or ignore the suggestions and continue drawing, thus maintaining the fluid control of drawing. They can also interactively refine the autocompleted results with a few tools. See Figure 1 for an example scenario.

The challenge of autocompletion is to predict suggestions that respect both users' inputs and the reference image. Our method is inspired by image analogy [Hertzmann et al. 2001] and operation history analysis and synthesis [Xing et al. 2014] while leveraging two key insights. First, since the act of drawing repetitive strokes usually indicates specific intentions (e.g., filling an object or hatching a shading region), we use the common image features among the coherent repetitive strokes to infer the intended regions. Second, the drawing usually relates to the underlying reference image (e.g., the density of strokes with respect to the image lightness). Therefore, we analyze the properties of both the drawing and the reference image to infer possible relationships as contextual constraints for stroke prediction.

Our system can reduce users' workload in drawing repetitive strokes over reference images and allow full control, similar to traditional manual drawing. We demonstrate its efficacy through various drawing results.¹

¹Please refer to our project page for more information: <https://yiroro.github.io/projects/auto-sketch/>.

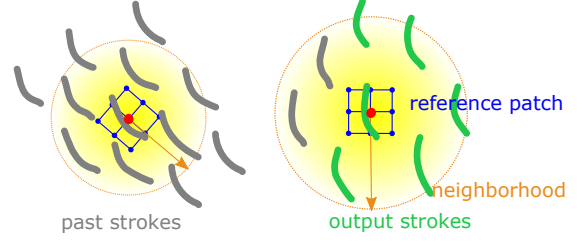


Figure 3: Illustration of our synthesis algorithm. We synthesize the predicted strokes (in green) from previously drawn strokes (in gray) by matching their neighborhoods. Each neighborhood includes the strokes within the yellow circles and the image patch (blue pixel grid) at the center. The neighborhood range (yellow circle radius) is determined by the radius map, and the neighborhood direction (yellow arrow) is determined by the orientation map.

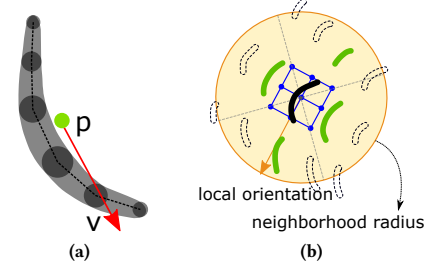


Figure 4: (a) A stroke, with centroid p and dominant direction v . (b) The neighborhood of the black stroke includes the n ($n = 1$ in this example) closest strokes (in green) from each quadrant and the middle image patch (blue pixel grid).

2 METHOD

Our autocomplete function involves two key algorithm steps: (1) inferring the input exemplar, the output region, and the contextual constraints from the stroke history and the reference image; (2) synthesizing suggestive strokes accordingly. This section first describes how to *synthesize* (Section 2.1) strokes, assuming all the information is available, and then explains how to *infer* (Section 2.2) the necessary information for synthesis.

2.1 Stroke Synthesis

The inputs to our stroke synthesis method include an exemplar E consisting of repetitive strokes, the reference image I , a target region mask M , an orientation map O , and a radius map R . Pixel values of R denote the extents of stroke spacing: a smaller value leads to a denser distribution. With the inferred information, this step aims to compute an output set of strokes X over the output region M , such that X is similar to E with respect to I . We extend the discrete element texture synthesis method [Ma et al. 2011; Xing et al. 2014] with an additional reference image, as depicted in Figure 3.

Stroke representation. As shown in Figure 4a, a stroke s is an ordered list of sample points, each with a timestamp and appearance attributes such as thickness and color. Here we focus on autocompleting short strokes, so we represent each stroke by its centroid

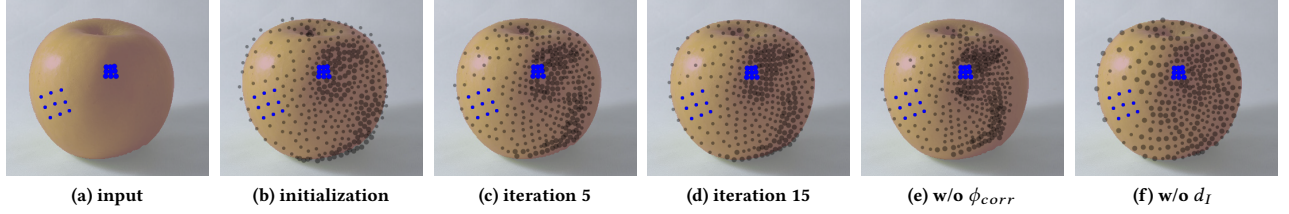


Figure 5: Iteration process in (b) to (d) and ablation studies in (e) and (f). Without the correction term the synthesized strokes tend to shift as in (e). Without the image term d_I the synthesized strokes might not follow the reference sufficiently as in (f).

p and the average direction v for efficiency during synthesis and reconstruct all the sample points accordingly after synthesis.

Initialization. We initialize the output X on the output region M with Poisson-disk sampling based on the radius map R . For each sampled position, we copy the input stroke with the smallest image feature distance d_I , which will be explained in Equation 2. We then optimize the output for a few objectives, as detailed below.

Neighborhood term. The neighborhood of a stroke consists of its neighboring strokes and the image patch around its centroid. As depicted in Figure 4b, we consider not only the spatial distance between strokes but also their locations when selecting the neighboring strokes, leading to a minimum representative neighborhood. For a stroke s and a neighboring stroke $s' \in N(s)$, we compute their difference in position and direction as:

$$\hat{u}(s', s) = \left(\frac{1}{R(s)} O(s)^{-1} (p(s') - p(s)), O(s)^{-1} (v(s') - v(s)) \right), \quad (1)$$

which is computed in the local frame defined by the radius map R and orientation map O . Therefore, the neighborhood distance between an output stroke s_o and an input stroke s_i is:

$$d_{neigh}(s_o, s_i) = \sum_{s'_o \in N(s_o)} \left| \hat{u}(s'_o, s_o) - \hat{u}(s'_i, s_i) \right|^2 + \mu \underbrace{|I(s_o) - I(s_i)|^2}_{d_I}, \quad (2)$$

where s'_i is the matched input stroke for s'_o via the Hungarian algorithm [Ma et al. 2011], the second term measures the image feature distance d_I , and $\mu (= 0.1)$ controls the relative weighting. The image feature is represented by the mean *Lab** color of an $R(s) \times R(s)$ patch at the stroke centroid. The overall neighborhood term to minimize is:

$$\phi_{neigh}(X, E) = \sum_{s_o \in X} \min_{s_i \in E} d_{neigh}(s_o, s_i). \quad (3)$$

Correction term. Since the neighborhood term only preserves the spatial distribution of strokes but not their alignment to the image and might result in undesired shifts (e.g., Figure 5e), we apply a correction term to address this issue. We compute a weighted centroidal Voronoi diagram from all the strokes' center points, using $\frac{1}{R}$ as weight, and denote the computed region centroids as $\{\bar{p}\}$. Thus we can minimize the distance between each output stroke centroid and the region centroid, defined as follows:

$$\phi_{corr}(X) = \sum_{s_o \in X} |p(s_o) - \bar{p}(s_o)|^2. \quad (4)$$

Solver. We iteratively minimize the energy function below following the EM methodology in [Ma et al. 2011].

$$\phi(X, E) = (1 - w)\phi_{neigh} + w\phi_{corr}. \quad (5)$$

Let m be the total number of iterations. For the i -th iteration, we set $w = (i/m)^2$, which means that more weight is given to ϕ_{neigh} in the beginning of iterations, so that we can optimize the neighborhood distribution first before doing corrections, which leads to better results. Figure 5b, 5c, 5d show the iterative optimization process of both the objectives.

2.2 Inference

2.2.1 Stroke exemplar E . This step aims to detect whether stroke repetitions exist and obtain the repetitive group as an exemplar for the synthesis process. We assume the example strokes to be temporally consecutive and have certain similar properties. Therefore, we start from the last stroke input by the user and search backward in the stroke sequence to incrementally find strokes that have similar shape and image features to the last stroke. Specifically, the stroke shape similarity is measured with the Fréchet distance, and the image features include *Lab** color and precomputed semantic segmentation at a stroke's center. We compare the standard deviation of a feature in the traversed k strokes against a threshold (15/255 for the color feature, 1 for the segmentation feature) for similarity measurement. The back-traversal stops when the next stroke does not contain any similar feature or $k > 50$. These k strokes serve as the input exemplar for the synthesis process. See Figure 6 for an example of the incremental searching process.

2.2.2 Output region M . Since the act of drawing repetitive strokes usually indicates specific intentions, we use the common image features among the coherent repetitive strokes to find a similar region for output. Because there are only two features in our implementation, we simply obtain the region by GrabCut if the *Lab** color feature is shared among the exemplar strokes, directly take the corresponding segmentation if the semantic feature is shared, and take the intersection if both features are shared. See Figure 6 for an example. When there are multiple disconnected regions, we retain the nearest region to the user's last stroke and discard the rest, because it is less natural to propagate to distant regions.

2.2.3 Contextual constraints. Since the drawing usually relates to the underlying reference image, we analyze the properties of both the drawn strokes and the reference image to infer possible relationships that control the global distribution of strokes.

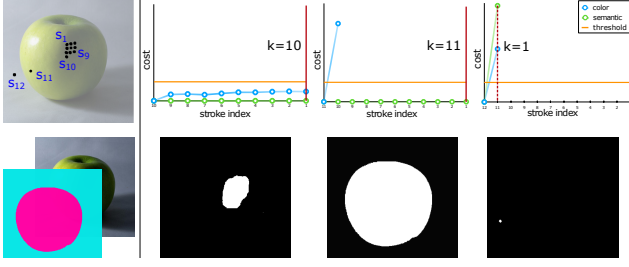


Figure 6: An example of predicting the input exemplar and output region. The left column shows the input stroke sequence visualized in black dots (only a few indices are shown for clarity) on the reference image (top) and the image features (bottom). The right columns show the threshold lines and the image feature cost curves for s_{10} , s_{11} , s_{12} respectively (top), and the corresponding predicted output regions (bottom). The cumulative number k is determined when both cost curves exceed the threshold.

Orientation O . Artists usually adjust the stroke directions to convey curvatures, but they may sometimes randomize or fix the stroke orientation regardless of the depicted objects to create different visual effects. Therefore, the problem is to decide which case the input exemplar implies. We first compute the edge tangent field (ETF) [Kyprianidis and Kang 2011] for the reference image and then calculate the angles between the exemplar strokes and the ETF directions at their centroids. If the standard deviation of the angles is small (less than 15 degrees), we consider the stroke orientations to be related to the ETF and take the ETF as the orientation field; otherwise, a default global coordinate frame is assigned.

Radius R . Since density is inversely proportional to the spacing between strokes, we reframe the problem as predicting a radius map that controls the extent of stroke neighborhoods. First, we compute the distance from each exemplar stroke to its nearest neighbor. We assume a linear relationship between these minimum distances r and the image features, including image lightness l and gradient strength g at a stroke's centroid, represented as: $r = (l \ g \ 1) \cdot t$, where t denotes the coefficients to solve. With the fitted linear model, if the squared correlation value is lower than 0.5 (the closer to 1, the better explanation), we use the model to compute a radius map. Otherwise, we consider the density as uniform and set the radius to the average spatial distance among the exemplar.

3 RESULTS AND DISCUSSION

We have presented a method to help users autocomplete repetitive short strokes with guidance from reference images while maintaining the flexible control of manual drawing. Figure 7 shows some sample drawings created with our system, where a lot of repetitive strokes are autocompleted to reduce manual effort.

However, our current approach still has a few limitations. First, we rely on simple *Lab** color and semantic segmentation for region inference. While color feature is sufficient for most cases, regions with similar colors but different semantics will require sufficient segmentation accuracy for region inference. Instead of precomputing the segmentation map, taking users' real-time input as additional cues for interactive segmentation might help improve the accuracy.

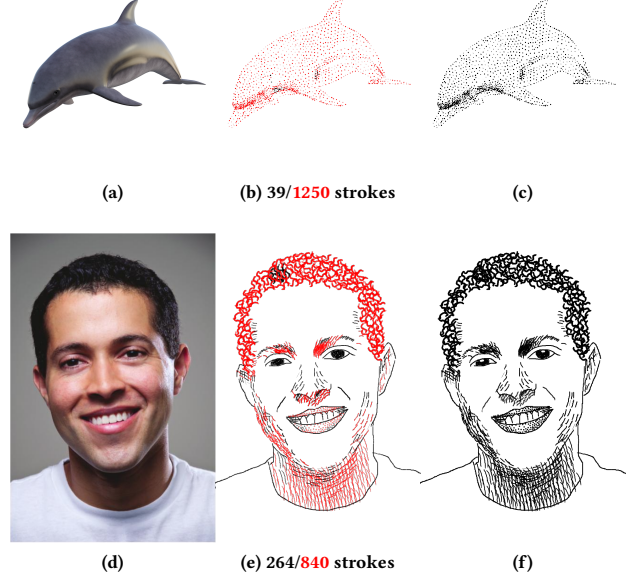


Figure 7: Sample results. The left column shows the reference images, the middle column visualizes the manual strokes (black) and autocompleted results (red) of the final drawings on the right column. Image credits: (a) Jazella, (d) Levy Carneiro Jr, under Creative Commons licenses.

Besides, our method only supports short discrete strokes, while artists also use continuous stroke textures [Dunn 2015]. It is worth investigating extending our framework with [Tu et al. 2020] to support continuous strokes.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for feedback, and funding from Adobe Research and the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation)-Project-ID 251654672-TRR 161.

REFERENCES

- Alphonso Dunn. 2015. *Pen and Ink Drawing: A Simple Guide*. Three Minds Press.
- Aaron Hertzmann, Charles E. Jacobs, Nuria Oliver, Brian Curless, and David H. Salesin. 2001. Image Analogies. In *SIGGRAPH '01*. 327–340.
- Yongcheng Jing, Yezhou Yang, Zunlei Feng, Jingwen Ye, Yizhou Yu, and Mingli Song. 2017. Neural style transfer: A review. *arXiv preprint arXiv:1705.04058* (2017).
- Rubaiat Habib Kazi, Takeo Igarashi, Shengdong Zhao, and Richard Davis. 2012. Vignette: Interactive Texture Design and Manipulation with Freeform Gestures for Pen-and-ink Illustration. In *CHI '12* (Austin, Texas, USA). 1727–1736.
- Jan Eric Kyprianidis, John Collomosse, Tinghui Wang, and Tobias Isenber. 2013. State of the "Art": A Taxonomy of Artistic Stylization Techniques for Images and Video. *IEEE Transactions on Visualization and Computer Graphics* 19, 5 (2013), 866–885.
- Jan Eric Kyprianidis and Henry Kang. 2011. Image and Video Abstraction by Coherence-Enhancing Filtering. *Computer Graphics Forum* 30, 2 (2011), 593–602. Proceedings Eurographics 2011.
- Chongyang Ma, Li-Yi Wei, and Xin Tong. 2011. Discrete Element Textures. *ACM Trans. Graph.* 30, 4, Article 62 (July 2011), 10 pages.
- Peihan Tu, Li-Yi Wei, Koji Yatani, Takeo Igarashi, and Matthias Zwicker. 2020. Continuous Curve Textures. *ACM Trans. Graph.* 39, 6, Article 168 (Nov. 2020), 16 pages.
- Jun Xing, Hsiang-Ting Chen, and Li-Yi Wei. 2014. Autocomplete Painting Repetitions. *ACM Trans. Graph.* 33, 6, Article 172 (Nov. 2014), 11 pages.